

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ

до лабораторної роботи №1
з дисципліни “ТППМП”

Студента 4 курсу
спеціальності 123 –
“Комп’ютерна інженерія”
Кузана Дениса Вікторовича

м. Ужгород – 2026 р.

Лабораторна робота №1

Хід виконання індивідуального завдання

Завдання 1 Створити програму для виводу консольного повідомлення «Hello, World»

Лістинг коду(Swift):

```
import Foundation

print("Hello, World!")
```

Лістинг коду(Objective-C):

```
#import <Foundation/Foundation.h>

int main(int argc, const char * argv[]) {
    @autoreleasepool {
        printf("Hello, World!");
    }
    return 0;
}
```

Результати виконання:

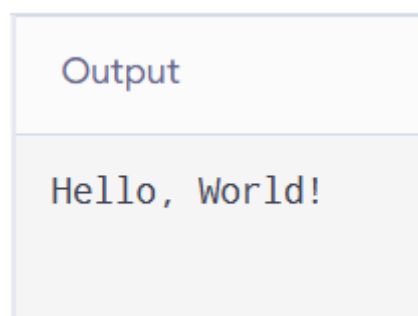


Рисунок 1 – Результат виконання завдання 1

Завдання 2 Описати сутності, які відповідають критеріям наведеним нижче:

1. Ціле десяткове число 12 з мінімально необхідною розрядною сіткою.
2. Ціле десяткове число -100 з мінімально необхідною розрядною сіткою.
3. Ціле шістнадцяткове число, що рівне цілому десятковому числу 128.
4. Мінімальне десяткове значення числа, яке знаходиться у межах 16-розрядної сітки.
5. Максимальне десяткове значення числа, яке знаходиться у межах 64-розрядної сітки.
6. Число з плаваючою крапкою 10,235.34 з мінімально необхідною розрядною сіткою.
7. Символ “a”.
8. Рядок “Hello World”.
9. Істина.
10. Число 12 та його рядкове представлення “twelve”.

Лістинг коду(Swift):

```
import Foundation
```

```
// 1. Ціле десяткове число 12 з мінімально необхідною  
розрядною сіткою (8-bit unsigned: 0...255)
```

```
let num12: UInt8 = 12
```

```
// 2. Ціле десяткове число -100 з мінімально необхідною  
розрядною сіткою (8-bit signed: -128...127)
```

```
let numNeg100: Int8 = -100
```

```
// 3. Ціле шістнадцяткове число, що рівне десятковому 128 (128  
у hex = 0x80)
```

```
let hexNum: UInt8 = 0x80
```

```
// 4. Мінімальне десяткове значення числа у межах 16-розрядної
сітки (-32,768)
let min16Bit: Int16 = Int16.min

// 5. Максимальне десяткове значення числа у межах
64-розрядної сітки (беззнакове: 18,446,744,073,709,551,615)
let max64Bit: UInt64 = UInt64.max

// 6. Число з плаваючою крапкою 10,235.34 з мінімально
необхідною розрядною сіткою (Float має 32 біти, precision ~6
знаків)
let floatNum: Float = 10235.34

// 7. Символ "a"
let charA: Character = "a"

// 8. Рядок "Hello World"
let strHelloWorld: String = "Hello World"

// 9. Істина
let isTrue: Bool = true

// 10. Число 12 та його рядкове представлення "twelve" (Кортеж
/ Tuple)
let numberAndName: (number: Int, name: String) = (12,
"twelve")
```

Завдання 3 Описати рядок “Hello World. This is Swift programming language” та виконати пункти, що описані нижче:

1. Визначити та вивести на екран довжину рядка.
2. Замінити кожне входження символу “i” на символ “u”.
3. Видалити 4-й, 7-й та 10-й символи.
4. Додати рядок “ (modified)” до існуючого рядка.
5. Вивести значення, що визначає чи існуючий рядок є пустим.
6. Додати символ “.” до кінця існуючого рядка.

7. Вивести значення, що визначає чи рядок починається з підрядка “Hello”.

8. Вивести значення, що визначає чи рядок закінчується підрядком “world.”

9. Вставити символ “-” після 10-го символу.

10.Замінити послідовність “Thus us” послідовністю “It is”

11.Вивести 10-й та 15-й символи існуючого рядка.

12.Вивести підрядок, що знаходиться у межах 10-го (включно) та 15-го (невключно) символів.

Лістинг коду(Swift):

```
import Foundation

// Початковий рядок
var text = "Hello World. This is Swift programming language"

// 1. Визначити та вивести на екран довжину рядка
print("1. Довжина рядка: \(text.count)")

// 2. Замінити кожне входження символу "i" на символ "u"
text = text.replacingOccurrences(of: "i", with: "u")
print("2. Після заміни 'i' -> 'u': \(text)")

// 3. Видалити 4-й, 7-й та 10-й символи (індекси 3, 6, 9)
// Видаляємо у зворотному порядку індексів, щоб не збивати
позиції
let indicesToRemove = [3, 6, 9].map {
    text.index(text.startIndex, offsetBy: $0) }
for index in indicesToRemove.sorted(by: >) {
    text.remove(at: index)
}
print("3. Після видалення 4, 7, 10 символів: \(text)")

// 4. Додати рядок " (modified)" до існуючого рядка
text += " (modified)"
print("4. Після додавання ' (modified)': \(text)")

// 5. Вивести значення, що визначає чи існуючий рядок є пустим
```

```

print("5. Рядок пустий: \(text.isEmpty)")

// 6. Додати символ "." до кінця існуючого рядка
text.append(".")
print("6. Після додавання крапки: \(text)")

// 7. Вивести значення, що визначає чи рядок починається з
підрядка "Hello"
print("7. Починається з 'Hello': \(text.hasPrefix("Hello"))")

// 8. Вивести значення, що визначає чи рядок закінчується
підрядком "world."
print("8.          Закінчується          на          'world.':
\ \(text.hasSuffix("world."))")

// 9. Вставити символ "-" після 10-го символу (тобто на
позицію з індексом 10)
let index10 = text.index(text.startIndex, offsetBy: 10)
text.insert("-", at: index10)
print("9. Після вставки '-': \(text)")

// 10. Замінити послідовність "Thus us" послідовністю "It is"
text = text.replacingOccurrences(of: "Thus us", with: "It is")
print("10. Після заміни 'Thus us' -> 'It is': \(text)")

// 11. Вивести 10-й та 15-й символи (індекси 9 та 14)
let char10Index = text.index(text.startIndex, offsetBy: 9)
let char15Index = text.index(text.startIndex, offsetBy: 14)
print("11. 10-й символ: '\(text[char10Index])', 15-й символ:
'\(text[char15Index])'")

// 12. Вивести підрядок у межах 10-го (включно, індекс 9) та
15-го (невключно, індекс 14) символів
let substringRange = char10Index..

```

Результати виконання:

```
Output

1. Довжина рядка: 47
2. Після заміни 'i' -> 'u': Hello World. Thus us Swift programming language
3. Після видалення 4, 7, 10 символів: Hello ord. Thus us Swift programming language
4. Після додавання ' (modified)': Hello ord. Thus us Swift programming language (modified)
5. Рядок пустий: false
6. Після додавання крапки: Hello ord. Thus us Swift programming language (modified).
7. Починається з 'Hello': false
8. Закінчується на 'world.': false
9. Після вставки '-': Hello ord. -Thus us Swift programming language (modified).
10. Після заміни 'Thus us' -> 'It is': Hello ord. -It is Swift programming language (modified).
11. 10-й символ: ' ', 15-й символ: 'i'
12. Підрядок з 10-го включно по 15-й невключно: ' -It '
```

Рисунок 2 – Результат виконання завдання 3

Завдання 4 Виконати пункти, описані нижче:

1. Описати сутність, що містить або не містить ціле десяткове число `integerNumber`, але не задає значення за замовчуванням.
2. Описати сутність, що містить або не містить число з плаваючою крапкою `decimalNumber`, але не задає значення за замовчуванням.
3. Присвоїти значення числу `integerNumber`.
4. Додати до числа `integerNumber` те ж значення, використовуючи `increment` та (або) `decrement` оператор.
5. Змінити знак числа на протилежний, використовуючи `unary minus` або `plus` оператор.
6. Присвоїти значення числу `decimalNumber` значенням числа `integerNumber`.
7. Описати сутність `pairOrValues`, що містить або не містить значення `integerNumber` та `decimalNumber`.

8. Перевірити, чи містить сутність `pairOfValues` цілочислове значення та вивести його, якщо таке значення існує.

9. Перевірити, чи містить сутність `pairOfValues` значення, що представлене число з плаваючою крапкою, та вивести його, якщо таке значення існує.

10. Вивести значення числа `decimalNumber` використовуючи `optional binding`.

Лістинг коду(Swift):

```
import Foundation

// 1. Сутність, що містить або не містить ціле десяткове число
integerNumber без значення за замовчуванням
var integerNumber: Int?

// 2. Сутність, що містить або не містить число з плаваючою
крапкою decimalNumber без значення за замовчуванням
var decimalNumber: Double?

// 3. Присвоїти значення числу integerNumber
integerNumber = 15

// 4. Додати до числа integerNumber те ж значення
(інкремент/додавання)
if let current = integerNumber {
    integerNumber = current + current
}

// 5. Змінити знак числа на протилежний, використовуючи unary
minus (-)
if let current = integerNumber {
    integerNumber = -current
}

// 6. Присвоїти значення числу decimalNumber значенням числа
integerNumber
if let intVal = integerNumber {
    decimalNumber = Double(intVal)
}
```

```
// 7. Опишіть сутність pairOfValues, що містить або не містить
// значення integerNumber та decimalNumber
var pairOfValues: (intVal: Int?, decimalVal: Double?)? =
(integerNumber, decimalNumber)

// 8. Перевірте, чи містить сутність pairOfValues цілочислове
// значення та виведіть його
if let pair = pairOfValues, let intVal = pair.intVal {
    print("8. pairOfValues містить integerNumber: \(intVal)")
} else {
    print("8. integerNumber відсутнє у pairOfValues")
}

// 9. Перевірте, чи містить сутність pairOfValues значення з
// плаваючою крапкою та виведіть його
if let pair = pairOfValues, let decVal = pair.decimalVal {
    print("9. pairOfValues містить decimalNumber: \(decVal)")
} else {
    print("9. decimalNumber відсутнє у pairOfValues")
}

// 10. Виведіть значення числа decimalNumber використовуючи
// optional binding (if let)
if let finalDecimal = decimalNumber {
    print("10. decimalNumber через optional binding:
    \(finalDecimal)")
} else {
    print("10. decimalNumber дорівнює nil")
}
```

Результати виконання:

Output
8. pairOfValues містить integerNumber: -30
9. pairOfValues містить decimalNumber: -30.0
10. decimalNumber через optional binding: -30.0

Рисунок 3 – Результат виконання завдання 4

Висновки: Під час виконання лабораторної роботи було здобуто практичний досвід створення консольних додатків мовами Swift та Objective-C. У ході роботи вивчено основи синтаксису мови Swift, систему типів з урахуванням розрядності даних, інструменти для роботи з рядковими величинами та індексами, а також безпечні методи опрацювання відсутніх значень за допомогою механізму Optionals (*Optional Binding*).